

CryptAgent

Presentation and study guide — the current AES source reviewer, its history, architecture, model, dataset, validation and next steps

Snapshot: 2 October 2026. This guide describes files in this local checkout. The active web app can change later; check the repository status before presenting.

1. A two-minute explanation you can read aloud

“CryptAgent is our BTP project for reviewing cryptographic integration code. The current demonstration focuses on one fixed case: C++20 code using Windows CNG for AES-128-GCM. We check five requirements: functional correctness, rejection of tampering, no plaintext release before authentication, correct key/nonce/tag handling, and safe input/output boundaries. The user pastes code into a local web page. A Node.js backend validates it, sends it only to a Llama model running on the same machine, checks the model’s structured response and source-line references, and returns an advisory report. A small deterministic source scanner adds clearly labeled observations. The pipeline runs, but this is source review, not native Windows execution or a security certificate. An experimental AES adapter was trained and compared with the base model; it improved output validity more than defect detection and is not deployed. CKKS, TFHE and BFV research fixtures support the planned multi-profile expansion.”

One-sentence version: CryptAgent currently turns a pasted CNG AES-GCM C++ excerpt into a local, source-linked review with explicit uncertainty.

2. What problem we are solving

A cryptographic algorithm may be sound while an application uses its API incorrectly. For example, a program may ignore an authentication failure, publish decrypted bytes too early, reuse a nonce with the same key, pass the wrong tag length, or advertise more output capacity than it allocated. CryptAgent concentrates on these integration mistakes. Its current threat model allows an attacker to modify encrypted input and observe public results, but not read the secret key.

The words verify and model need care in the presentation. Today the browser delivers an advisory source-code review. It does not compile the submitted code, execute attacks, mathematically prove correctness, or confirm that a snippet is deployable. A model’s “no issue identified” means only that the model found none in the visible text.

3. AES-128-GCM in practical terms

AES is a symmetric block cipher: encryption and decryption use the same secret key. GCM is an authenticated encryption mode. It gives confidentiality through encryption and an authentication tag to detect changes to the ciphertext and any associated authenticated data (AAD). The fixed CryptAgent profile uses a 16-byte AES key (128 bits), a 12-byte nonce and a 16-byte tag.

Conceptually, encryption takes key + fresh nonce + plaintext + optional AAD and returns ciphertext + tag. Decryption takes key + nonce + ciphertext + AAD + tag and must check the tag before accepting plaintext. AAD is bound to the message but is not itself encrypted. A nonce must not be repeated for encryption under the same key. A zero-initialized nonce buffer alone is not proof of reuse: the code may fill it with a fresh value before encryption.

On Windows CNG, code selects the AES algorithm and GCM chaining mode, creates a key handle, initializes `BCRYPT_AUTHENTICATED_CIPHER_MODE_INFO`, and supplies its nonce, AAD and tag fields to

BCryptEncrypt or BCryptDecrypt. The return status matters: a tag mismatch is an authentication failure. CryptAgent reviews the caller's handling of these fields and the result; the current Linux service does not run CNG itself.

Part	What the reviewer checks
Key / 16 bytes	Does the visible integration meet the AES-128 profile and handle key creation or import safely?
Nonce / 12 bytes	Is each encryption nonce fresh under the same key, and is the correct size passed to CNG?
Tag / 16 bytes	Is a tag supplied and checked with the configured length?
AAD	When the application contract requires AAD, is the same data authenticated on both paths?
Status and output	Does failed decryption return failure without exposing plaintext?

4. Fixed scope and five requirements

The active profile is aes128-gcm-tampering-v1 in profiles/aes128_gcm_tampering.json. The page displays AES-128-GCM, C++20 / Windows CNG and message tampering. There is no scheme, library, attacker or language selector in this midterm flow.

ID	Requirement	What a relevant source finding could look like
F01	Functional correctness	A supported input, including a documented boundary case, is handled incorrectly.
F02	Tampering rejection	The wrapper fails to reject a modified ciphertext, nonce, tag or required AAD.
F03	Authentication before output	The status is ignored or caller output is populated before authentication succeeds.
F04	Key, nonce and tag handling	Wrong profile size, repeated encryption nonce or missing tag configuration.
F05	Input/output boundaries	Unchecked length conversion, underflow, or a CNG capacity larger than the buffer.

The profile assumes a trusted operating system and CNG implementation, an attacker without the secret key, and observable public status/output. Replay prevention, physical/timing side channels and universal memory-safety proofs are outside this version.

5. What was built before the current web app

The repository preserves the original Phase 1–3 research: requirements and threat model; machine-readable schemas and a C++ contract; synthetic cases, a harness, and validation tools. Those historical files use a different profile: libsodium XChaCha20-Poly1305, with 13 contract requirements. A historical 150-case benchmark represents 30 scenario groups with shared ancestry; it is not 150 independent implementations. The older Windows validation document reports package/schema checks and a syntax-only C++ header check, while native libsodium candidate execution was not run because the dependency was missing. These facts are research history, not claims about the active AES/CNG web service.

The present AES-only scope came later. Broader Rust/Python FHE, lattice and multi-scheme experiments remain stored in the repository or source folders. They are not selectable in the current app and are not suitable labels for this fixed AES/CNG reviewer.

6. Architecture: how the pieces connect

```
Browser  http://127.0.0.1:8000
  | GET /api/status -> profile + model state + session token
  | POST /api/verify -> code + profile ID + verify mode
  v
Node backend (validation, source observations, report policy)
  | GET /health + POST /review on fixed loopback address
  v
Python service  http://127.0.0.1:8081
  | local cached Llama-3.1-8B-Instruct on CUDA GPU
  v
JSON answer -> strict validator -> report -> browser / JSON export
```

The connection is an HTTP request inside the same machine. The backend hardcodes `127.0.0.1:8081`; user input cannot select a remote model URL. The Python service and web server bind to loopback. There is no OpenAI API key, paid API call or normal external transmission of submitted source in this active path. The services do not persist submitted source, although an exported report contains quoted excerpts.

7. Frontend: what the student uses

`frontend/index.html` defines the fixed-profile screen. `frontend/styles.css` lays out the editor and result panel. `frontend/app.js` handles interaction.

- Status: asks `GET /api/status` for the profile, model health and a per-process session token; loading/error status refreshes automatically.
- Input: paste code or open a C++ source/header file; the editor shows line numbers and byte/line counts. File upload is local browser reading, not a cloud upload.
- Review: sends JSON with code, fixed profile ID and mode `verify`. During review the editor locks and shows progress.
- Result: shows a headline, five requirement assessments, code quotations, clickable source line locations, model message and limitations. Export JSON downloads the report on the user's machine.
- Errors: invalid or empty input, service failure and timeout are displayed explicitly. Unrelated code can receive "outside scope"; short snippets can receive "needs context."

8. Node backend: each module's job

File	Responsibility
backend/app.mjs	Serves the page; issues a random session token; accepts only status and verify API routes; checks host/origin, JSON shape, profile, source limits and concurrency.
backend/verifier.mjs	Runs a narrow pattern scanner, calls the model client, derives an overall advisory verdict, and constructs the version 2.0 report.
backend/llm_client.mjs	Calls the fixed local Python endpoint, verifies service/model/revision/profile/prompt identity, bounds response size and time, and applies JSON validation.
backend/review_contract.mjs	Requires the five F01-F05 assessments, consistent scope/findings and exact quotations in valid source lines.
backend/native_checks.mjs	Marks every native check not_run; it does not execute user C++.

The source-pattern scanner currently notices such text as a discarded BCryptDecrypt call, a standalone decrypt call, a zero initializer near a nonce, some output-related identifiers and an ECB mode identifier. These are observations, not confirmed security defects. The model's own findings are labeled separately.

Input is limited to nonempty source with no null bytes, at most 64 KiB and 2,000 lines. The backend allows one active review. It returns explicit HTTP errors for malformed requests or busy status. The model client has a 110-second deadline, and the browser has a 125-second deadline. Invalid model JSON is rejected rather than silently treated as a pass.

9. Local Llama: loading and inference

backend/local_model.py runs a Python HTTP service with GET /health and POST /review. It loads the cached meta-llama/Llama-3.1-8B-Instruct base model at a pinned revision, in offline mode. PyTorch uses the NVIDIA CUDA GPU; Transformers and bitsandbytes load the model in 4-bit form with bfloat16 computation. This saves GPU memory. No AES adapter is loaded into the active service.

For each request, Python combines the fixed review instructions, the fixed profile, and line-numbered untrusted source text into a Llama chat prompt. It rejects prompts above 3,072 input tokens without truncation. Generation is deterministic in configuration (do_sample=False) and bounded by 1,536 new tokens or 90 seconds. A generation that stops at a time/token limit without an end token is marked incomplete. One review runs at a time. The service reports loading, ready, busy and error states.

The local model is an advisory reviewer. Its JSON must contain a scope, exactly five checks and source-linked findings. The Node validator confirms formatting, cross-field consistency and quoted source location. It cannot determine whether every explanation is cryptographically correct. That is why fine-tuning, expert label review and independent tests matter.

10. One code submission, step by step

Step	What happens
1	The page obtains the fixed profile, local model state and session token.
2	The student pastes/uploads source and presses Review AES code.
3	Node checks the request and normalizes line endings; it hashes the submitted source.
4	A small scanner records source-pattern observations. Node asks local Llama to review the same source.
5	Python returns JSON; Node validates five IDs, scope, status consistency and exact source quotations.
6	Node combines validated model assessments and source observations into an advisory report.
7	The page shows the report and can export its JSON. The candidate was not compiled or run.

Example: a snippet calls BCryptDecrypt, discards its return status and then returns success. The source scanner can flag the discarded-status line as a concern for F03. That observation invites review of authentication failure handling; it is not a native tampering witness.

11. How to read a report

The report records a profile ID/version/hash, source hash and line count, model identity and prompt hash, overall verdict, the five per-check assessments, findings with source lines and quotations, and explicit execution/formal-verification statuses. The possible overall verdicts are:

Verdict	Meaning
potential_issues	Source observations or model findings need human review.
no_issues_identified	The model reported no concerns on visible source; not a proof or runtime pass.
insufficient_context	The relevant implementation or caller context is missing.
outside_scope	The model classified the submission outside the fixed Windows CNG AES-GCM profile.
review_unavailable	Model unavailable, busy, incomplete or invalid; no model conclusion accepted.

Each requirement can say potential issue, no issue identified, insufficient context, or not applicable for outside-scope code. A model issue without a valid source quotation is downgraded. The overall “potential issues” headline can come from a source observation even if the model wrongly says “no issue” on a specific check; the report shows both so the discrepancy is visible.

12. Dataset and experimental fine-tuning

Earlier project datasets include different cryptographic schemes and languages. Those are useful historical research materials but do not match the fixed AES/CNG runtime contract. A newer candidate AES source-review release is stored in `training/aes_source_pilot_v1/data`: 22 train, 4 validation and 4 test = 30 unique code samples. It contains 16 distinct short training snippets plus six related full-session variants; the rest are development checks. The examples cover ignored authentication status, early plaintext publication, nonce/key/tag sizes, nonce handling, buffer capacity, out-of-scope code and missing context.

The release stores expected scope, F01-F05 assessments, explanations and exact source-linked quotations. The builder checked normalized duplicates, family membership across splits, production JSON validation, prompt hashes and the 3,072-token limit. The longest serialized training sample was 3,049 tokens. The training labels were authored from source review; no Windows native run validates them.

Current training status: an explicitly experimental two-epoch AES LoRA pilot used 22 training rows and four validation rows, leaving four test rows untouched. Its manifest still says `training_ready=false` because independent full-session label review and a fresh held-out split remain outstanding. The active web app still uses the cached base Llama model, not this adapter.

The 20-case synthetic comparison is stored in `evaluation/aes_pilot_20/REPORT.md`. Fully accepted reviews improved from 7/20 to 20/20, but correctly located targeted concerns were only 0/13 for the base and 3/13 for the adapter; both avoided false alarms on 3/4 targeted controls. This is exploratory source review, not a generalization or security-accuracy claim.

The trainer (`training/train.py`) performs offline assistant-output-only QLoRA fine-tuning with explicit dataset selection and hashes. It normally refuses draft releases; a restricted `--experimental-draft` option was used for this AES pilot. Mixed-scheme records, truncated examples and existing output directories remain guarded. An adapter requires independent evaluation and explicit deployment before affecting the app.

13. What has actually been tested

For the AES web stack, 11 Node tests and four Python HTTP tests passed. They cover fixed routes, profile/input guards, local-only model calls, malformed or contradictory JSON, fake source quotations, offline/error behavior and response structure. Three synthetic live GPU smoke submissions returned complete reports: unrelated Python was outside scope; declarations needed more context; an ignored-decryption-status excerpt showed potential issues. These checks establish that the pipeline works, not that its security predictions are accurate.

The base model missed the ignored-status concern on an individual F03 assessment; the source-pattern observation surfaced the overall potential issue. It also made optimistic “no issue” assessments for incomplete excerpts. A stricter prompt experiment made responses less reliable and was reverted. This is direct evidence for requiring better labels and a proper evaluation before presenting accuracy claims.

There is currently no native Windows machine for CNG runtime verification. The active report explicitly marks candidate execution and formal verification `not_run`. The legacy `VALIDATION.md` records older Phase 1-3 checks on a historical Windows host; those are not current AES/CNG runtime results.

14. Setup and machine notes

On the current Linux laboratory machine, the application uses the existing Python environment at `/home/hp/crypto-llm/.venv/bin/python` and a working NVIDIA RTX A4000 GPU with 16 GiB of memory. A CUDA tensor test succeeded. The app starts with `bash start.sh`, which launches both Node and Python; Node serves port 8000 and Python serves 8081 on 127.0.0.1. The app also includes `start.ps1` for Windows, but the active CNG runtime was not verified on Windows in this work.

The repository ignores local model weights, adapters, checkpoints, executables and environments so they are not committed to ordinary Git history. It does not use the OpenAI API. If the cached Llama weights, CUDA environment or local Python dependencies are missing, the page stays available and reports model loading/unavailable instead of inventing a review result.

15. How to demonstrate it honestly

- Start: run `bash start.sh` in the repository, then open `http://127.0.0.1:8000` and wait for “Local Llama is ready.”
- Show fixed settings: point to AES-128-GCM, C++20 / Windows CNG and message tampering. Explain why a fixed profile makes the present evaluation tractable.
- Paste a small AES CNG excerpt: include a visible `BCryptDecrypt` call whose status is discarded. Review it, click the cited line and show the F01–F05 list.
- Show uncertainty: submit a declaration-only excerpt or unrelated code and show “insufficient context” or “outside scope.” A model failure can also be shown as review unavailable.
- Explain the boundary: point to “candidate execution: not run” and state that the model is still a base model. Avoid describing the finding as a proved exploit.

If asked for a “secure/not secure” answer: “Today the report gives source-review concerns and uncertainty. A confirmed pass/fail would require independent labels plus native CNG compilation and tamper tests on Windows.”

16. Likely supervisor questions and concise answers

Question	Answer
Why GCM?	It combines encryption with authentication; the integration must handle nonce uniqueness, AAD, tag checking and failure status correctly.
Why local Llama?	Submitted code stays in the local workflow and no paid API is required. The model can explain source concerns, but its opinions still need validation.
Is the model trained?	The active model is the cached Llama-3.1-8B-Instruct base model. The 30-code AES candidate is prepared; training has not begun.
Why only AES?	The current contract and five checks are fixed so data, prompts, UI and reports can be evaluated consistently. Multi-scheme work is historical/future scope.
Can it prove code secure?	No. It checks source text and marks native execution not run. We need Windows CNG fixtures and independent evaluation for stronger claims.
What is the next experiment?	Finish expert label review, obtain a fresh held-out AES set, measure baseline errors, train an adapter, then compare on untouched data.

17. Next steps and success criteria

First, review the full-session AES labels and build a fresh evaluation set by implementation family, including correct code, real misuse, suspicious-but-valid cases, incomplete context and out-of-scope code. Then measure base-model validity, per-check defect recall/precision, false positives, abstention, quoted-line accuracy and explanation quality. After the dataset is approved, run QLoRA locally and repeat the same evaluation. Only if the adapter improves relevant safety metrics should it be explicitly activated in the Python service. Separately, test Windows CNG code on a Windows machine with synthetic keys/messages and tampered ciphertext, nonce, tag and AAD. Keep source observations separate from confirmed runtime witnesses.

18. File map and sources

Read first: README.md, STATUS.md, docs/AES_REVIEW_CONTRACT.md. Current code: frontend/index.html, frontend/app.js, frontend/styles.css; backend/app.mjs, backend/verifier.mjs, backend/llm_client.mjs, backend/review_contract.mjs, backend/native_checks.mjs, backend/local_model.py; profiles/aes128_gcm_tampering.json; prompts/verify.txt; tools/start_local.mjs; start.sh/start.ps1. Data: training/aes_source_pilot_v1/data/manifest.json and train/validation/test.jsonl. History: phase1/, phase2/, phase3/, WALKTHROUGH.md, VALIDATION.md.

Primary API references: [Microsoft BCryptEncrypt](#); [Microsoft BCryptDecrypt](#); [Microsoft authenticated-cipher parameters](#). Model identification: [Llama-3.1-8B-Instruct model card](#). Project-specific claims above were checked against the local repository snapshot and dataset manifest.

19. Quick glossary for the presentation

Term	Meaning in CryptAgent
AEAD	Authenticated encryption with associated data: encryption plus a tamper-detection tag.
AAD	Data authenticated with a message but not encrypted, such as agreed metadata.
Nonce	A per-encryption value that must not repeat under the same AES-GCM key.
Tag	The authentication value checked during decryption; 16 bytes in this profile.
CNG	Windows Cryptography Next Generation, the intended native cryptographic API.
Inference	Running the already-loaded Llama model on submitted source; this is not training.
QLoRA / LoRA	An efficient fine-tuning method that trains small adapter weights on a quantized base model; used experimentally here.
Held-out set	Code examples excluded from training and reserved to measure behavior after training.
Source observation	A deterministic text-pattern match that points to code for human inspection.
Native witness	A future Windows CNG test result tied to an actual compiled candidate and input.

20. Short command and status checklist

```
cd /home/hp/Documents/Codex/2026-09-29/pull-and-setup/cryptAgent
bash start.sh
# Open http://127.0.0.1:8000 and wait for Local Llama is ready.
# Automated contracts: npm test; python3 tests/test_local_model.py
# Candidate dataset counts: training/aes_source_pilot_v1/data/manifest.json
```

Before presenting, say explicitly: “The local pipeline runs; the active base model is experimental; the 30-code candidate has not been used for training; submitted C++ has not been compiled or executed.” This single statement prevents the historical tests, source observations and model opinions from being confused with a completed cryptographic verification.